BML MUNJAL UNIVERSITY

School of Engineering and Technology

Course: MTH2802 – Linear Algebra & Differential Equations

B.Tech Computer Science & Engineering, Semester III

Project Report

Predicting Population Movement Using Transition Matrices

A Markov-chain model of movement between Delhi, Gurugram, Noida and Faridabad

Group 14

Name	Roll No.
Mansi Awasthi	250450
Prince Kumar Yadav	250500
Shobhit Rai	250557
Ishank Chauhan (Group Representative)	250406

Instructor: Dr. Luckshay Batra

November 2026

Abstract

In this project we model how people move between four regions of the Delhi–NCR area (Delhi, Gurugram, Noida and Faridabad) using a column-stochastic transition matrix P, with one time step equal to one year and the update rule xk+1 = P xk. The starting populations are Census 2011 district totals, while the transition probabilities are **simulated** because public inter-city annual flow data are not available at this level. We verify that P is regular, compute future distributions through matrix powers, and find the long-term (steady-state) distribution in three independent ways: the eigenvector for λ = 1, Gaussian elimination on (P − I)π = 0, and a high matrix power. All three agree to within 10^{-8}. The steady state is Delhi 55.5%, Gurugram 17.5%, Noida 16.7% and Faridabad 10.3%. The eigenvalues are real and distinct (1, 0.8862, 0.8638, 0.8600), so P can be diagonalised, and |λ2| = 0.8862 explains the convergence speed (32 years to come within 1%; the eigen-decomposition predicts 33). We study two what-if scenarios and a 1000-run sensitivity test. Finally we extend the model to an open system with births, deaths and migration (xk+1 = A xk + m) and fit it to the 2001 and 2011 censuses. This shows that the simulated P moves about seven times too many people, and it projects about 36.8 million people in the four districts by 2041. Finally we build P from real data, the Census 2011 migration table D-02: real movement between the regions is about 100 times smaller than our simulated P, so the closed model would need thousands of years to settle, while an open model fitted to the census with Newton's method reproduces Delhi's net migration from an independent source and projects about 33.7 million people by 2041.

1 Introduction and Problem Statement

People in a metropolitan region keep moving between cities for work, housing and family reasons. If, each year, a fixed fraction of the people living in one city move to another, then the future population of every city can be predicted with simple matrix multiplication. This is the idea of a **transition matrix**, and it is a good first application of the linear algebra we study in MTH2802.

The official brief for this project is: construct a transition matrix representing movement of people between selected regions using simulated or publicly available data; use matrix powers to estimate future population distributions and study the long-term behaviour of the model. We choose four regions of the National Capital Region (NCR): Delhi, Gurugram, Noida and Faridabad.

1.1 Objectives

1. Construct a 4 × 4 column-stochastic transition matrix P for the four regions and verify its properties.
2. Use repeated multiplication and matrix powers P^k to project the population for k = 0 to 30 years.
3. Find the long-term steady state in three independent ways and check that they agree.
4. Diagonalise P ($P = V D V^{-1}$) and use the eigenvalues, especially $|\lambda_2|$, to explain how fast the model converges.
5. Study scenarios (what-if changes) and the sensitivity of the answer to small errors in P.
6. State the limitations of the model, including what happens when P is not regular.
7. Extend the model to an open system (births, deaths, migration) and test the simulated P against Census 2001 and 2011.
8. Build P from real Census 2011 migration data and check the model against an independent source.

1.2 Convention used

We follow the textbook of Poole and use **column-stochastic** matrices. The entry P[i][j] is the probability that a person living in region j this year lives in region i next year, so every column sums to 1. If x_k is the population vector after k years, then $x_{k+1} = P\, x_k$. The order of regions is always Delhi, Gurugram, Noida, Faridabad.

2 Mathematical Background

2.1 Stochastic matrices and Markov chains

A square matrix P is **column-stochastic** if all its entries are non-negative and every column sums to 1. A **Markov chain** is a process that moves between a finite number of states where the probability of the next state depends only on the present state (the memoryless property). Here the states are the four regions and the transition probabilities are stored in P.

Why columns sum to 1. Column j lists what happens to a person living in region j: they must live somewhere next year, either in the same region or in one of the others. The probabilities of all these possibilities add up to 1. Consequently the total population is unchanged by $x_{k+1} = P\,x_k$ (our model is a closed system: nobody is born, dies, enters or leaves the four regions).

2.2 Why λ = 1 is always an eigenvalue

Let **1** be the vector of all ones. Since each column of P sums to 1, we have $\mathbf{1}^T P = \mathbf{1}^T$, which means $P^T\mathbf{1} = \mathbf{1}$. So 1 is an eigenvalue of P^T. A matrix and its transpose have the same characteristic polynomial because $\det(P - \lambda I) = \det(P^T - \lambda I)$, so 1 is also an eigenvalue of P. Moreover, every eigenvalue λ of a stochastic matrix satisfies $|\lambda| \le 1$.

A useful by-product: if v is an eigenvector of P for an eigenvalue $\lambda \neq 1$, then $\mathbf{1}^T P v = \mathbf{1}^T v$ gives $(1 - \lambda)(\mathbf{1}^T v) = 0$, so the entries of v add up to 0. Eigenvectors for $\lambda \neq 1$ therefore describe redistributions of population, not population itself.

2.3 Regular chains and the steady-state theorem

P is **regular** if some power P^k has all entries strictly positive (i.e. after k steps it is possible to get from any region to any other). **Steady-state theorem (stated, not proved):** if P is regular, then (i) λ = 1 is a simple eigenvalue and all other eigenvalues satisfy $|\lambda| < 1$; (ii) there is a unique probability vector π with $P\pi = \pi$; and (iii) for any starting distribution x_0 with total 1, $x_k = P^k x_0 \to \pi$ as $k \to \infty$. Moreover P^k approaches the matrix whose every column is π.

Intuition: if every region can reach every other region, the process keeps mixing the population. Memory of the starting point fades and only the balance between inflow and outflow remains. At the steady state, the number leaving each region equals the number arriving, so the shares stop changing.

2.4 Diagonalisation and P^{n} = V D^{n} V^{-1}

If P has n linearly independent eigenvectors $v_1, \ldots, v_n$ with eigenvalues $\lambda_1, \ldots, \lambda_n$, put them as columns of V and the eigenvalues on the diagonal of D. Then $P = V D V^{-1}$. Since the inner $V^{-1}V$ cancel, $P^n = V D^n V^{-1}$, and D^n is easy because we only raise each diagonal entry to the power n. Distinct eigenvalues guarantee that P is diagonalisable, which is why we arranged the matrix so that all eigenvalues are real and distinct.

Write the starting vector as $x_0 = c_1v_1 + c_2v_2 + \ldots + c_nv_n$, where $c = V^{-1}x_0$. Then

$$x_k = c_1(1)^k v_1 + c_2\lambda_2^k v_2 + c_3\lambda_3^k v_3 + \ldots$$

The terms with $|\lambda_j| < 1$ die out as k grows, so only c_1v_1 survives, and that is the steady state. The speed of convergence is controlled by the largest of the remaining $|\lambda_j|$, namely $|\lambda_2|$: the closer it is to 1, the slower the convergence, and the distance to the steady state shrinks roughly like $|\lambda_2|^k$.

2.5 A worked 2 × 2 example

Let T = [[0.9, 0.2],[0.1, 0.8]] (two cities; 10% of city 1 move to city 2 each year and 20% of city 2 move to city 1). Columns sum to 1.

- **Eigenvalues:** the characteristic equation is $\lambda^2 - (\text{trace})\lambda + \det = \lambda^2 - 1.7\lambda + 0.70 = 0$, so λ = 1 and λ = 0.7.

- **λ = 1:** (T − I)v = 0 gives −0.1a + 0.2b = 0, so a = 2b and v_1 = (2, 1). Normalising to sum 1 gives the steady state π = (2/3, 1/3).
- **λ = 0.7:** (T − 0.7I)v = 0 gives 0.2a + 0.2b = 0, so v_2 = (1, −1) (entries sum to 0, as predicted).
- **Diagonalisation:** V = [[2, 1],[1, −1]], D = diag(1, 0.7). Hence $T^n = V D^n V^{-1} \to$ [[2/3, 2/3],[1/3, 1/3]] as $n \to \infty$ because $0.7^n \to 0$.
- **Check:** starting from (0.5, 0.5) the code gives (0.5000, 0.5000) → (0.5500, 0.4500) → (0.5850, 0.4150) → (0.6095, 0.3905), moving towards (0.6667, 0.3333).

2.6 Open systems: births, deaths and migration

A closed model keeps the total population fixed for ever. To let people be born, die, leave the NCR or arrive from the rest of India we use the affine model xk+1 = A xk + m, with A = diag(1 + g) P diag(1 − e). Here ej is the share of region j that leaves the NCR each year, gi is the natural growth rate (birth rate minus death rate) and m is the yearly number of arrivals. The columns of A no longer sum to 1, so A is not stochastic.

Homogeneous coordinates. Writing the state as (xk, 1) turns the affine step into a linear one: (xk+1, 1) = B (xk, 1) with B = [[A, m],[0, 1]], the same trick used in computer graphics. B is block upper-triangular, so its eigenvalues are those of A together with 1.

Closed form. If 1 is not an eigenvalue of A, put $x^* = (I - A)^{-1}m$. Then $x_k = A^k(x_0 - x^*) + x^*$. If every $|\lambda(A)| < 1$ the population settles at x^*; otherwise A^k dominates and the population keeps growing.

Perron–Frobenius. A has only positive entries, so its largest eigenvalue ρ(A) is real, simple and has a positive eigenvector. In the long run the total grows by a factor ρ(A) per year and the shares approach that eigenvector (normalised to sum 1). This replaces the steady state π of the closed model.

Scaling mobility. For $0 < \alpha \le 1$ let $P_\alpha = I + \alpha(P - I)$. Its columns still sum to 1 and $P_\alpha \pi = \pi$, so the steady state is unchanged, but every eigenvalue moves to $1 + \alpha(\lambda - 1)$: fewer movers give the same destination, reached more slowly. We use α to fit the model to the census in Section 7.1.

3 Data and Assumptions

3.1 Regions and initial population

We use four regions at district level. The initial populations in Table 1 are the **official district totals from the Census of India 2011** (Gurugram = Gurgaon district, Noida = Gautam Buddha Nagar district). The total is 21,760,221. An earlier draft used approximate values for Gurugram (1,514,085) and Noida (1,674,714); both were wrong and have been corrected. All code inputs are in one module (inputs.py), so changing a figure needs one edit and a re-run.

Table 1: Initial populations (Census of India 2011, district totals)

Region	Unit used	Population (Census 2011)	Initial share
Delhi	NCT of Delhi	16,787,941	77.15%
Gurugram	Gurgaon district	1,514,432	6.96%
Noida	Gautam Buddha Nagar district	1,648,115	7.57%
Faridabad	Faridabad district	1,809,733	8.32%

3.2 Simulated transition probabilities

The transition probabilities in this project are simulated. Public data on yearly movement between individual NCR cities, in the form of a complete origin–destination matrix, are not available at this granularity (Census migration tables are collected only occasionally, by state or district, and use different time windows). We therefore designed P by hand using plain-language reasoning about how the four

places behave, and we kept it within the rules we set ourselves: retention between 0.88 and 0.96, all entries strictly positive, columns summing to 1, and real distinct eigenvalues. Table 2 explains each column.

Table 2: Justification of each column of P (simulated)

Column (from)	Stay	Where the movers go	Plain-language reasoning
Delhi	0.950	Gurugram 0.020, Noida 0.020, Faridabad 0.010	Highest retention: the largest job market, established housing and family ties. Movers prefer Gurugram and Noida (IT and corporate jobs) over Faridabad.
Gurugram	0.890	Delhi 0.060, Noida 0.030, Faridabad 0.020	Many workers are recent arrivals with weaker roots; some return to Delhi for housing cost, family or education. Few move to Faridabad.
Noida	0.890	Delhi 0.060, Gurugram 0.030, Faridabad 0.020	Similar to Gurugram (IT and job-driven residents). Slightly more movement back to Delhi, which is adjacent.
Faridabad	0.880	Delhi 0.070, Gurugram 0.030, Noida 0.020	Lowest retention: some residents move to Delhi (closest large job market) and Gurugram (jobs along the same corridor), fewer to Noida.

3.3 Assumptions

- Time step is one year and P is constant in time (the same every year).
- The system is closed: only the four regions exist; no births, deaths, or movement to or from the rest of India.
- The process is memoryless: where a person goes next depends only on where they live now.
- Each person in a region has the same probabilities (no age, income or occupation differences).
- Populations are Census 2011 district totals; the year 2011 is only the starting point of the simulation, and the closed model does not claim real forecasts (the open model in Section 7 is a first step towards that).
- In the sensitivity study, '±2%' means each entry of P is multiplied by a random factor in [0.98, 1.02] and each column is then renormalised to sum to 1.

3.4 Extra data for the open model (Section 7)

The open model needs three more inputs per region (Table 1b). The 2001 populations are Census 2001 totals; Faridabad's 2001 district still included Palwal (split off in 2008), so we back-compute its comparable value from the official 2001–2011 growth of 32.54%. Natural growth is the **Sample Registration System (SRS) 2020** birth rate minus death rate for the state (Delhi, Haryana, Uttar Pradesh), applied to each district. The emigration rates are **simulated**. Section 7.5 adds measured flows and arrivals from Census table D-02 and SRS rates for 2001–2010.

Table 1b: Inputs of the open model

Region	Census 2001	Natural growth per year (SRS 2020)	Leave the NCR per year (simulated)
Delhi	13,850,507	1.06% (14.2 − 3.6 per 1000)	0.6%
Gurugram	870,539	1.38% (Haryana)	0.8%
Noida	1,202,030	1.86% (Uttar Pradesh)	0.8%
Faridabad	1,365,424 (comparable)	1.38% (Haryana)	0.7%

4 Model Construction

The final transition matrix (rows = region next year, columns = region this year) is shown in Table 3 and as a heat map in Figure 1.

Table 3: Final transition matrix P (simulated), $x_{k+1} = P\,x_k$

to \ from	Delhi	Gurugram	Noida	Faridabad
Delhi	0.950	0.060	0.060	0.070
Gurugram	0.020	0.890	0.030	0.030
Noida	0.020	0.030	0.890	0.020
Faridabad	0.010	0.020	0.020	0.880

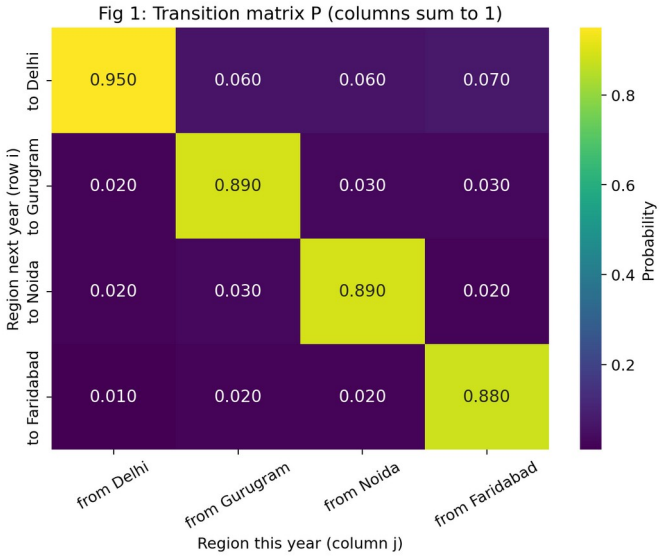


Figure 1: Heat map of the transition matrix P. Large diagonal values show that most people stay where they are.

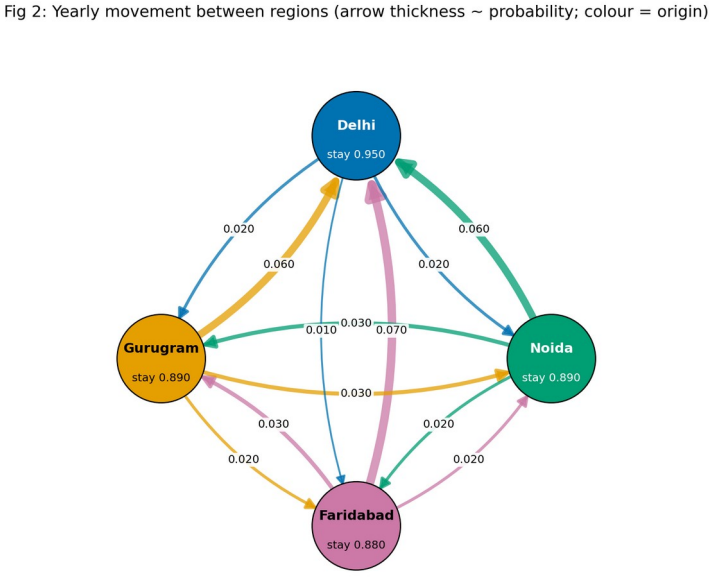


Figure 2: Flow diagram of the four regions. Arrow thickness is proportional to the transition probability.

4.1 Verification of properties

Table 4: Properties of P checked by the program (43 checks passed, 0 design warnings)

Property	Result
Every column sums to 1	Yes (column sums = 1.000, 1.000, 1.000, 1.000)
All entries non-negative	Yes
Retention (diagonal) between 0.88 and 0.96	k = 1 (P itself is strictly positive); the program searches up to Wielandt's bound $(n - 1)^2 + 1 = 10$, so a 'not regular' answer would be a proof
Regular: smallest k with $P^k > 0$	k = 1 (already P itself is strictly positive)
Eigenvalues real and distinct	Yes (1.0000, 0.8862, 0.8638, 0.8600)
No region below 5% in steady state	Yes (smallest share 10.32%)

The program separates two kinds of check. **Mathematical facts** (columns sum to 1, $P\pi = \pi$, the three methods agree, $P = VDV^{-1}$) must always hold; a failure stops the program. **Design choices** (retention between 0.88 and 0.96, every steady share above 5%, distinct eigenvalues) are properties of our particular P; if someone edits P and one of them fails, the program prints a warning instead of crashing.

5 Results

5.1 Population projections

Starting from the Census 2011 populations x_0, we computed $x_{k+1} = P\,x_k$ repeatedly for 30 years. Table 5 gives the population counts and Table 6 the shares for selected years.

Table 5: Projected population (persons) after k years

k	Delhi	Gurugram	Noida	Faridabad
0	16,787,941	1,514,432	1,648,115	1,809,733
1	16,264,978	1,787,339	1,884,209	1,823,695
2	15,799,681	2,027,268	2,092,339	1,840,933
5	14,690,023	2,587,145	2,580,106	1,902,947
10	13,522,696	3,153,225	3,076,056	2,008,244
20	12,515,807	3,613,964	3,480,205	2,150,245
30	12,207,751	3,746,048	3,594,629	2,211,793

Table 6: Projected share of total population after k years

k	Delhi	Gurugram	Noida	Faridabad
0	77.15%	6.96%	7.57%	8.32%
1	74.75%	8.21%	8.66%	8.38%
2	72.61%	9.32%	9.62%	8.46%
5	67.51%	11.89%	11.86%	8.75%
10	62.14%	14.49%	14.14%	9.23%
20	57.52%	16.61%	15.99%	9.88%
30	56.10%	17.22%	16.52%	10.16%

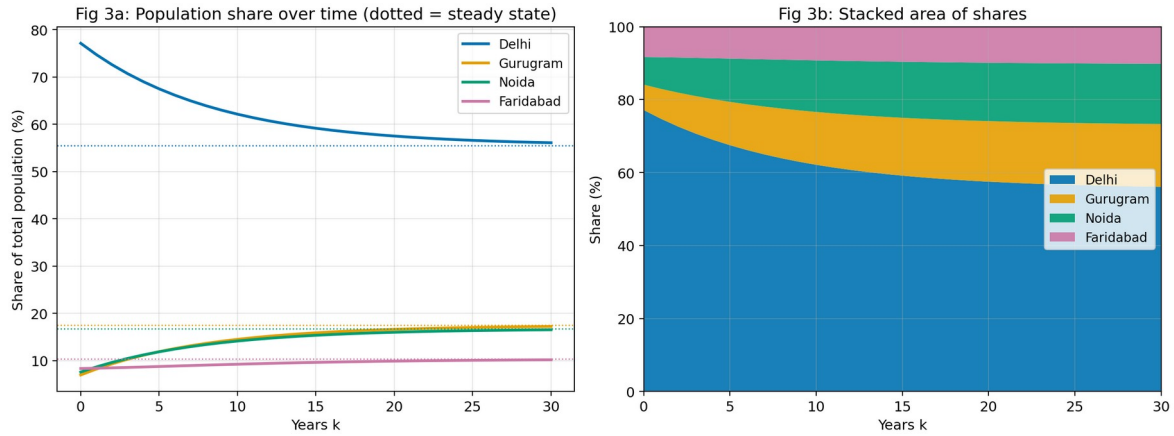


Figure 3: Population shares over 30 years (left: lines with dotted steady-state levels; right: stacked area).

The total stays at 21,760,221 in every year because P is column-stochastic. Delhi's share falls from 77.1% to 56.1% after 30 years, while Gurugram, Noida and Faridabad grow. A hand calculation for one entry is shown in Appendix B.

5.2 Matrix powers

Instead of repeating the step, we can compute P^k directly. Table 7 shows P^{10} and P^{50}. In P^{10} the columns are still different, but in P^{50} each row is nearly constant: all four columns have become almost the same vector. The largest gap between columns in any row is 0.310 for P^{10} and 0.0027 for P^{50}. That common column is the steady state.

Table 7: Selected matrix powers

Power	Delhi col	Gurugram col	Noida col	Faridabad col
P^{10} Delhi	0.6898	0.3796	0.3796	0.4097
Gurugram	0.1227	0.3742	0.1529	0.1514
Noida	0.1203	0.1490	0.3703	0.1218
Faridabad	0.0671	0.0972	0.0972	0.3171
P^{50} Delhi	0.5559	0.5533	0.5533	0.5540
Gurugram	0.1742	0.1755	0.1750	0.1748
Noida	0.1670	0.1676	0.1681	0.1671
Faridabad	0.1028	0.1036	0.1036	0.1040

5.3 Steady state: three independent methods

- **(a) Eigenvector for λ = 1,** normalised so its entries sum to 1.
- **(b) Linear system.** We solve $(P - I)\pi = 0$ together with $\pi_1+\pi_2+\pi_3+\pi_4 = 1$ by Gaussian elimination: the last row of P − I is replaced by a row of ones and the right-hand side by (0, 0, 0, 1).
- **(c) High power.** Every column of P^{200} is the steady state.

Table 8: Steady state from the three methods

Method	Delhi	Gurugram	Noida	Faridabad
(a) Eigenvector	0.55483871	0.17465438	0.16728111	0.10322581
(b) Linear system	0.55483871	0.17465438	0.16728111	0.10322581
(c) Column of P^{200}	0.55483871	0.17465438	0.16728111	0.10322581
Steady state (%)	55.48%	17.47%	16.73%	10.32%

The largest difference between the methods is 1.52e-11, far below the tolerance 10^{-8} that we required. The steady-state counts are Delhi 12,073,413, Gurugram 3,800,518, Noida 3,640,074, Faridabad 2,246,216. As a check of the balance idea, at the steady state the share leaving Delhi each year (0.05 × 0.5548 = 0.0277) equals the share arriving in Delhi (0.06×0.1747 + 0.06×0.1673 + 0.07×0.1032 = 0.0277).

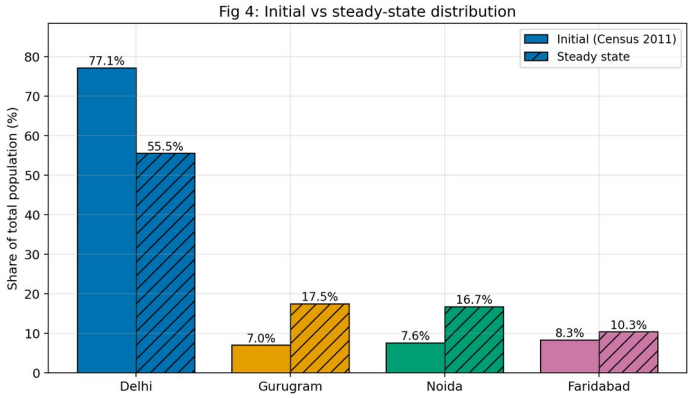


Figure 4: Initial (Census 2011) and steady-state shares.

5.4 Eigen-analysis and diagonalisation

The eigenvalues of P are all real and distinct: $\lambda_1 = 1.000000$, $\lambda_2 = 0.886180$, $\lambda_3 = 0.863820$, $\lambda_4 = 0.860000$ (Figure 9, now drawn in the complex plane so that complex or negative eigenvalues would also be visible). Hence P is diagonalisable: $P = V D V^{-1}$ with $D = \text{diag}(\lambda_1, \ldots, \lambda_4)$ and V holding the eigenvectors (Table 9). The reconstruction error $\max|V D V^{-1} - P|$ is 5.6e-16. For n = 10, the matrix $V D^{10} V^{-1}$ agrees with the direct power P^{10} to within 2.0e-15.

Table 9: Eigenvector matrix V (first column normalised to sum 1; the others to length 1)

	v_1 (λ=1)	v_2	v_3	v_4
Delhi	0.5548	0.8615	0.1750	0.0000
Gurugram	0.1747	-0.3290	-0.4582	0.7071
Noida	0.1673	-0.2034	0.7413	-0.7071
Faridabad	0.1032	-0.3290	-0.4582	0.0000

Writing the starting share vector as $x_0 = c_1v_1 + c_2v_2 + c_3v_3 + c_4v_4$ gives $c_1 = 1.0000$, $c_2 = 0.2841$, $c_3 = -0.1602$, $c_4 = -0.1202$. The coefficient c_1 equals 1 because the other eigenvectors sum to 0 and x_0 sums to 1. After 30 years the factors λ_j^{30} are 1.00e+0, 2.66e-2, 1.24e-2, 1.08e-2: the λ = 1 component stays at full strength, while the others have shrunk to about 1–3% of their start. This is the algebraic reason for convergence.

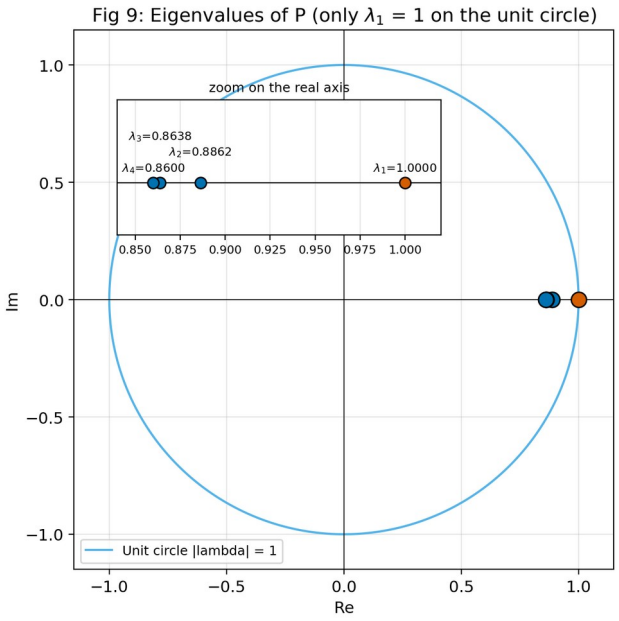


Figure 9: Eigenvalues of P in the complex plane with the unit circle (inset: zoom on the real axis). Only λ1 = 1 lies on the circle; the other three are inside it.

5.5 Convergence speed

We measure the distance to the steady state by $d_k = \Sigma|x_{k,i} - \pi_i|$ using shares. Because $d_k \approx C|\lambda_2|^k$, taking logarithms gives the estimate $k \approx \ln(\varepsilon/C)/\ln|\lambda_2|$ for the years needed to reach distance ε. With $|\lambda_2| = 0.8862$ and $C = \|c_2 v_2\|_1 = 0.4894$, taken from the eigen-decomposition ($c = V^{-1}x_0$), we get Table 10. An earlier draft fitted C from the simulated curve itself ($C = d_{20}/|\lambda_2|^{20} = 0.4558$), which made the 'theory' column partly circular. The slowest mode loses half of its size every 5.7 years.

Table 10: Years until the distance to the steady state drops below a threshold

Threshold	Actual (simulation)	Theory from $\|\lambda_2\|^k$
1% ($d_k < 0.01$)	32	33
0.1% ($d_k < 0.001$)	51	52

Theory over-estimates by one year because the faster modes ($\lambda_3 = 0.8638$, $\lambda_4 = 0.8600$) have coefficients of opposite sign to c_2, so they slightly cancel the slowest mode during the first decades.

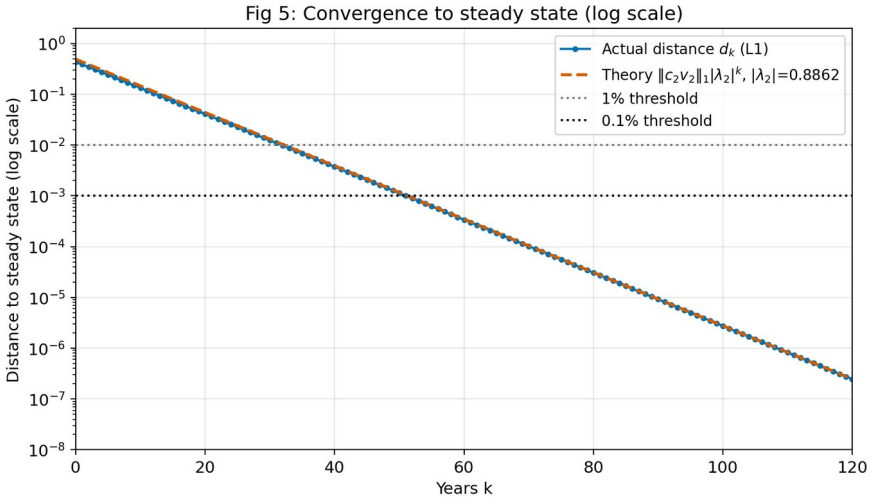


Figure 5: Distance to the steady state on a log scale. The straight line shows geometric decay at rate $|\lambda_2|$.

Different starting points. Figure 6 starts the system from four very different distributions (the Census-based one, everyone in Delhi, uniform, everyone in Faridabad). All of them end at the same steady state, as the theorem predicts: the long-term result depends on P, not on the starting point.

Fig 6: Different starting points converge to the same steady state

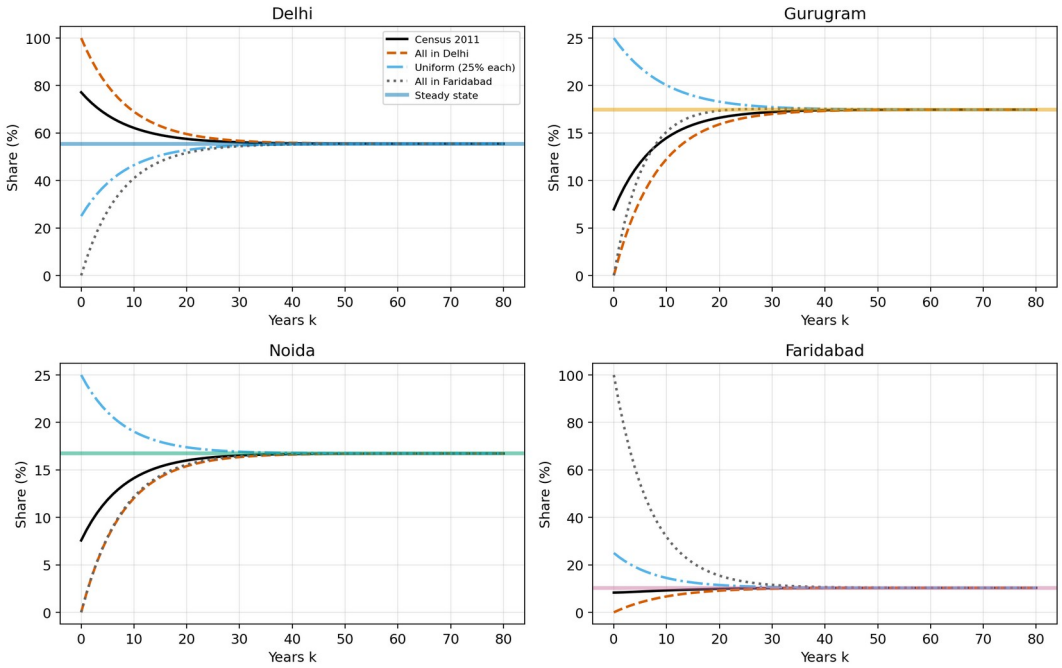


Figure 6: Four different starting distributions converge to the same steady state.

6 Scenario and Sensitivity Analysis

6.1 Scenarios (simulated what-if changes)

- **Scenario A – Faridabad improves infrastructure.** Faridabad's retention rises from 0.880 to 0.920, and the flows into Faridabad increase (from Delhi 0.010 → 0.015, from Gurugram 0.020 → 0.030, from Noida 0.020 → 0.030). The other columns are rebalanced so that every column still sums to 1.
- **Scenario B – Job boom in Gurugram and Noida.** Delhi's movers to Gurugram and Noida rise from 0.020 and 0.020 to 0.035 each (Delhi's retention drops to 0.920), and Gurugram and Noida each retain more people (0.920). Columns are rebalanced.

Table 11: Steady-state shares and $|\lambda_2|$ under each scenario

Scenario	Delhi	Gurugram	Noida	Faridabad	$\lvert\lambda_2\rvert$
Baseline	55.48%	17.47%	16.73%	10.32%	0.8862
A: Faridabad	46.62%	16.60%	15.86%	20.91%	0.9015
B: Job boom	34.15%	28.47%	27.53%	9.85%	0.8950
Change A (pp)	-8.86	-0.86	-0.87	+10.59	0.0153
Change B (pp)	-21.33	+11.00	+10.80	-0.48	0.0088

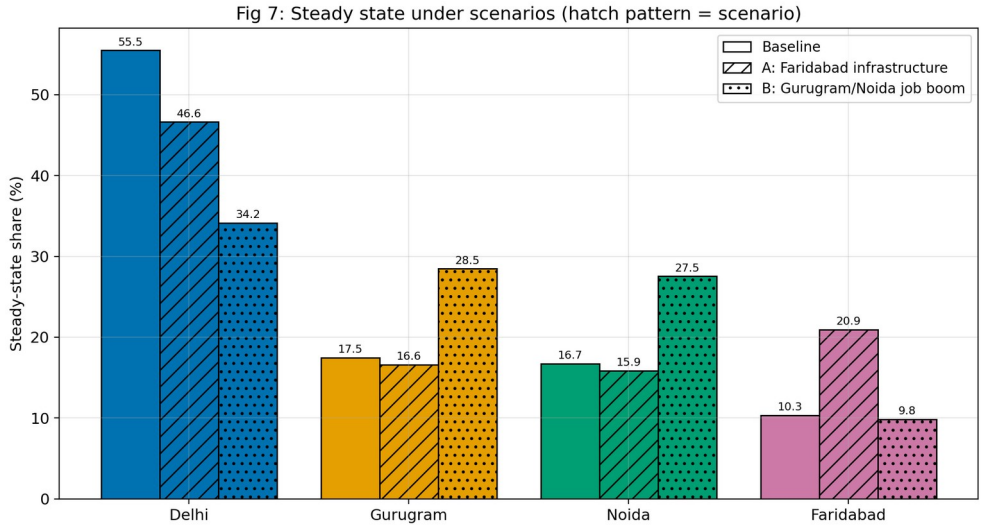


Figure 7: Steady-state shares under the baseline and the two scenarios.

In scenario A Faridabad's long-term share rises from 10.3% to 20.9%, mostly at the expense of Delhi (55.5% → 46.6%). In scenario B Gurugram and Noida jump to 28.5% and 27.5%, while Delhi falls to 34.2%. In both scenarios $|\lambda_2|$ rises slightly (0.8862 → 0.9015 and 0.8950), so the new steady state would be approached a little more slowly. Small changes in yearly probabilities therefore produce large changes in the long-run shares.

6.2 Sensitivity analysis

To test how much our conclusions depend on the exact numbers in P, we ran 1000 random experiments. In each run every entry of P was multiplied by a random factor between 0.98 and 1.02, the columns were renormalised, and the steady state was recomputed (random seed 42).

Table 12: Steady-state share (%) over 1000 perturbed matrices

Region	Baseline	Mean	Std. dev.	Min	Max
Delhi	55.48	55.47	0.39	54.36	56.68
Gurugram	17.47	17.48	0.24	16.74	18.20
Noida	16.73	16.73	0.23	15.94	17.49
Faridabad	10.32	10.33	0.14	9.95	10.74

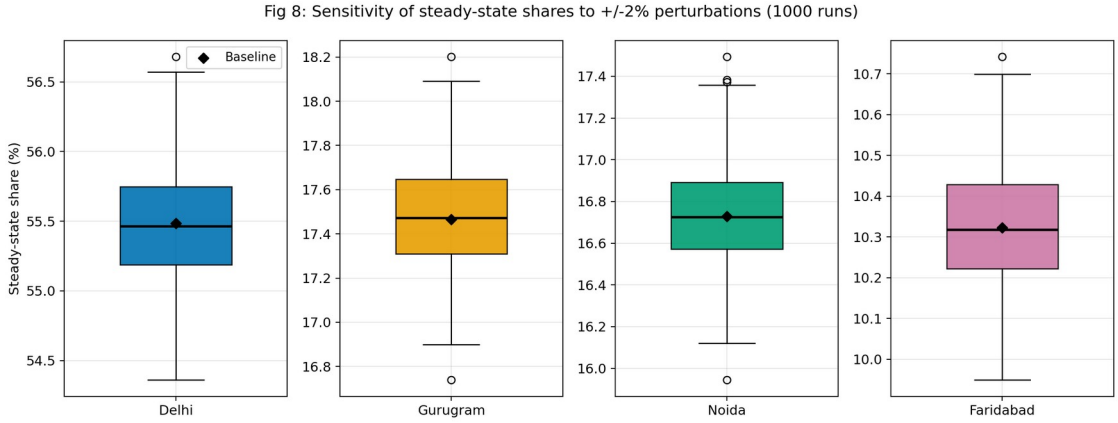


Figure 8: Distribution of the steady-state shares over 1000 perturbations (diamond = baseline).

Conclusion on robustness. The standard deviation of each share is at most 0.39 percentage points, and the largest deviation from the baseline over all runs is 1.20 percentage points. $|\lambda_2|$ stays between 0.8835 and

0.8894. So the qualitative picture (Delhi largest, Faridabad smallest, convergence in several decades) is robust to ±2% errors in P. It is, however, not robust to the larger structural changes of the scenarios, which is why accurate data on retention would matter.

7 Open-System Extension

Sections 4–6 used a closed system. Here we let people be born, die, leave the NCR and arrive from outside (Section 2.6), and we check the simulated P against two real censuses.

7.1 Can the simulated P explain the census?

Over 10 years the model gives $x2011 = A^{10}x2001 + S10m$ with $S10 = I + A + \ldots + A^{9}$. Since the censuses fix x2001 and x2011, the yearly arrivals m solve the 4 × 4 linear system $S10m = x2011 - A^{10}x2001$, which we solve with the same Gaussian elimination as in Section 5.3.

Finding. With P at full strength the solution needs **negative** arrivals: −210,414 people a year into Gurugram and −192,075 into Noida (Figure 10a). That is impossible, so our simulated P moves too many people. For example, 2% of Delhi moving to Gurugram is about 335,000 people a year, which would add 22% to Gurugram in a single year, while the census shows +74% over ten years. Scaling mobility with $P\alpha = I + \alpha(P - I)$, the largest α that gives non-negative arrivals everywhere is **α = 0.137**: the census supports only about 14% of the simulated movement. The calibrated arrivals are Delhi 293,386, Gurugram 21,143, Noida 0 and Faridabad 31,852 people a year (346,381 in total).

The steady state π does not change with α, so every result of Sections 5–6 about the long-run shares still holds. What changes is the speed: $|\lambda_2|$ becomes $1 + 0.137(0.8862 - 1) = 0.984$, so convergence is about 7.7 times slower. Note that this is an exact fit (four equations, four unknowns), not an independent test of the model.

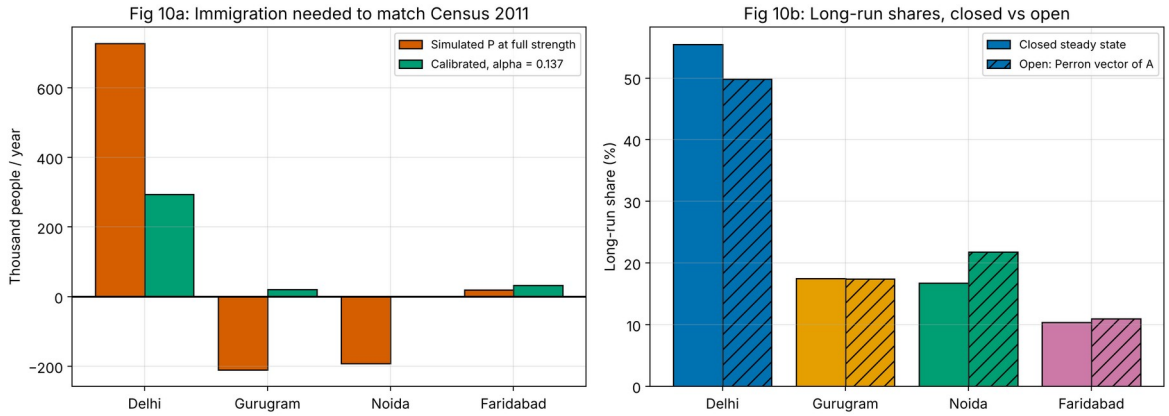


Figure 10: (a) Yearly arrivals needed to reproduce Census 2011, with P at full strength and with α = 0.137. (b) Long-run shares of the closed model (steady state) and the open model (Perron vector of A).

7.2 Forecast 2011–2041

Starting from Census 2011 with α = 0.137 and the calibrated arrivals, the four districts grow from 21.76 million to about 36.8 million in 2041 (Table 13, Figure 11). Unlike the closed model, Delhi grows too. Writing $x_{30} = A^{30}x_0 + S_{30}m$ splits the 2041 population into 25.6 million descendants of the 2011 residents and 11.2 million people who arrived after 2011, with their children (Figure 12).

Table 13: Open-model forecast (millions of people)

Year	Delhi	Gurugram	Noida	Faridabad	Total
2011	16.79	1.51	1.65	1.81	21.76
2021	19.80	2.22	2.19	2.28	26.49
2031	22.89	2.98	2.84	2.78	31.49
2041	26.08	3.80	3.59	3.32	36.78

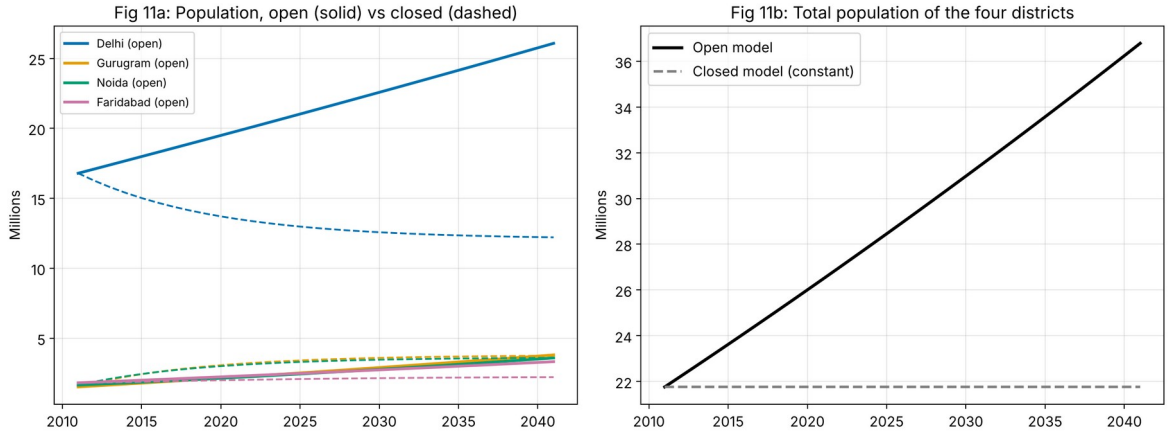


Figure 11: Open model (solid) against the closed model (dashed). Left: each region. Right: total population.

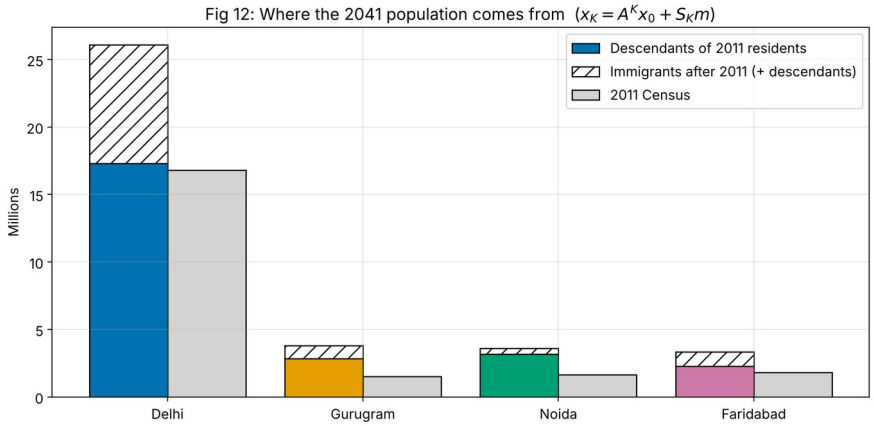


Figure 12: Where the 2041 population comes from: descendants of 2011 residents ($A^{30}x_0$) and later arrivals ($S_{30}m$).

7.3 Long-run behaviour

The dominant eigenvalue of A is ρ(A) = 1.00626, so in the long run the population grows by 0.63% a year. ρ(A) > 1, so $x^* = (I - A)^{-1}m$ is not an attractor (its entries are even negative); the shares instead approach the Perron vector of A: Delhi 49.8%, Gurugram 17.4%, Noida 21.8%, Faridabad 11.0% (Figure 10b). Noida ends higher than in the closed model (16.7%) because Uttar Pradesh has the highest natural growth. Because α is small, this limit is approached very slowly: in 2041 Delhi still holds 70.9%.

7.4 What-if cases

Table 14: Total population of the four districts in 2041 under different assumptions

Case	Total 2041 (million)	Delhi share	Change vs baseline
Baseline	36.78	70.9%	–
Emigration x2	31.23	71.9%	-5.56 M
Natural growth -30%	33.64	71.6%	-3.14 M
Immigration +50%	42.37	71.9%	+5.59 M
Immigration stops	25.60	67.6%	-11.18 M

Arrivals matter most: without them the four districts would hold 25.6 million people in 2041 instead of 36.8 million. Doubling emigration removes about 5.6 million. These numbers rest on simulated emigration and on α fitted to one decade, so they show sensitivities, not an official forecast.

7.5 Real migration data: Census 2011 table D-02

Our P was simulated because no yearly origin–destination matrix is published. The Census does publish table **D-02**, which counts migrants by their place of last residence and by how long they have lived where they were counted. People who had lived in a district for **less than one year** moved there during the year before the census, so they measure yearly flows. Dividing each flow by the population of its origin gives a census-based matrix Pc.

Flows from Delhi to each suburb are read directly. For the other directions D-02 only names the origin state (for example 'from Haryana'), so we split those flows in proportion to population, with a proximity factor w (w = 1: no extra pull from nearby districts). Table 15 gives the result.

Table 15: Yearly flows between the four regions from Census 2011 D-02 (people per year, w = 1)

to \ from	Delhi	Gurugram	Noida	Faridabad
Delhi	–	1,048	822	1,252
Gurugram	10,906	–	115	593
Noida	5,896	105	–	126
Faridabad	5,365	226	86	–

Finding. Real movement is far smaller than in our simulated P: the off-diagonal entries of P are 31 to 431 times larger than those of Pc (median 111 times). Only about 0.06–0.13% of each region moves to another of the four regions in a year, not 5–12% (Figure 13). This agrees in direction with Section 7.1, where the census totals already required much less movement than P.

For the closed model this changes the time scale completely. The eigenvalues of Pc are 1, 0.99927, 0.99876 and 0.99803, so $|\lambda 2| = 0.99927$, a half-life of about 945 years, and about 3,398 years to come within 1% of the steady state (32 years with the simulated P). The steady state itself is not pinned down by the data: Delhi's long-run share ranges from 32% to 66% as w goes from 1 to 4. The closed steady state is therefore a statement about the mathematics, not a forecast.

Fig 13: Yearly movement between regions, simulated vs census data

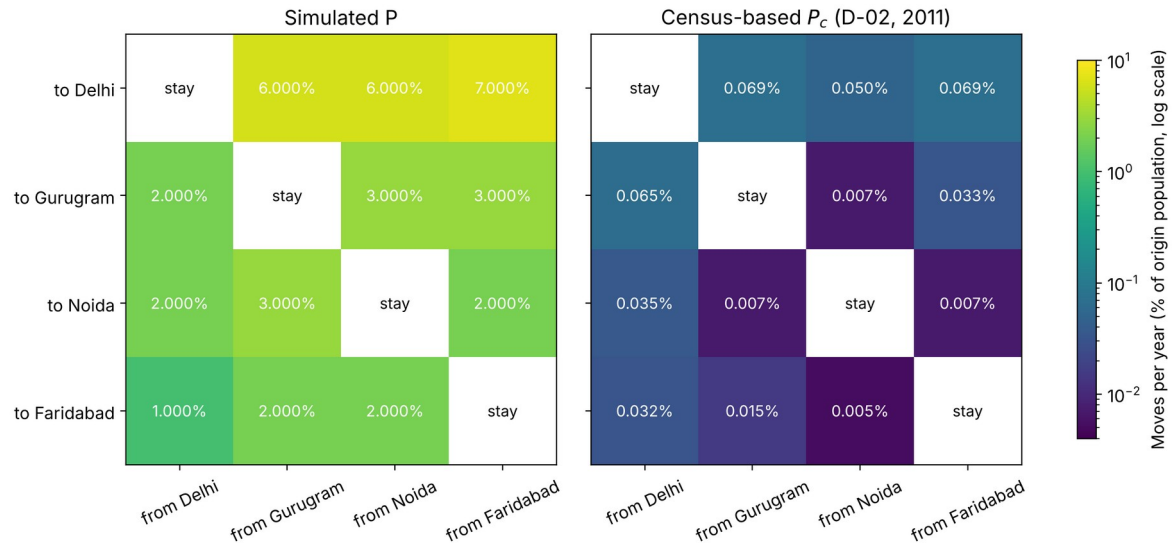


Figure 13: Yearly movement between the four regions as a percentage of the origin population (log colour scale): simulated P (left) and census-based Pc (right).

7.6 Open model on census data, and an independent check

We now rebuild the open model of Section 7.1 from measurements. The arrivals m from outside the four regions also come from D-02 (Delhi 244,268, Gurugram 55,133, Noida 45,658, Faridabad 24,374 people a year). Natural growth uses the SRS averages for 2001–2010 (Delhi 1.34%, Haryana 1.76%, Uttar Pradesh 2.13% a year), the decade we fit to. Only emigration is unknown.

Newton's method. x2011 depends non-linearly on the four emigration rates e, so we solve f(e) = x10(e) − x2011 = 0 iteratively: $e \leftarrow e - J^{-1}f(e)$, where the Jacobian $J_{ik} = \partial x_{10,i}/\partial e_k$ is found by finite differences and each step is a 4 × 4 linear system solved by Gaussian elimination. It converges in 5 iterations to e = 0.88% (Delhi), 1.75% (Gurugram), 2.49% (Noida) and 0.68% (Faridabad) a year. Gurugram and Noida need the most emigration partly because state-wide birth rates overstate the growth of these urban districts.

Independent check. The Economic Survey of Delhi 2014–15 estimates Delhi's yearly net migration from registered births and deaths and the change in population. None of these numbers was used to build the model. Table 16 and Figure 14 compare them: the model gives 0.93 lakh a year on average, the survey 0.69 lakh. The two agree best in 2010–2011, the years the D-02 flows were measured. Most of the gap comes from the birth and death data: the survey's registration data (CRS) give a natural increase of 14.7 per 1000, the SRS 13.4, which alone accounts for 0.20 of the 0.24 lakh gap.

Table 16: Net migration into Delhi (lakh per year): census-data model and Economic Survey of Delhi 2014–15

Year	Model	Survey	Year	Model	Survey
2002	1.06	0.55	2007	0.92	0.77
2003	1.03	0.63	2008	0.89	0.78
2004	1.00	0.61	2009	0.86	0.67
2005	0.97	0.56	2010	0.83	0.81
2006	0.94	0.69	2011	0.80	0.81

Fig 14: Independent check of the census-data model

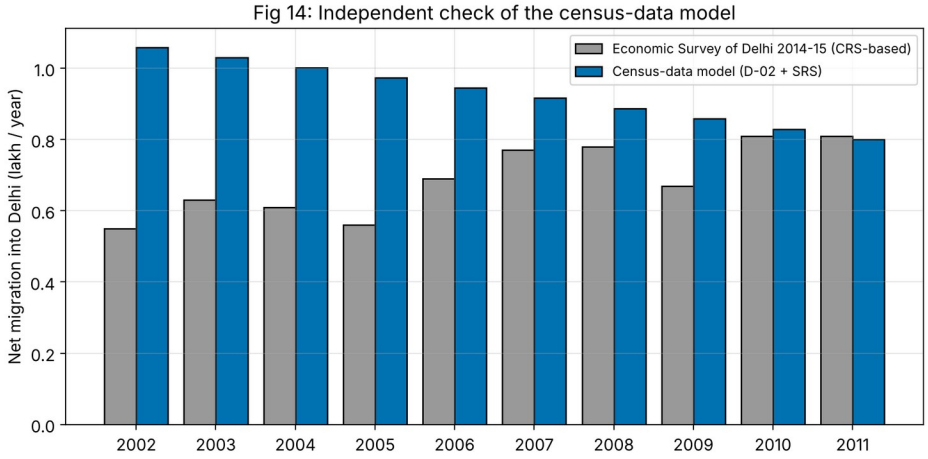


Figure 14: Net migration into Delhi, census-data model against the Economic Survey of Delhi 2014–15.

7.7 Forecast with census data

With the fitted emigration, the measured arrivals and today's SRS growth rates, the census-data model gives 25.7 million people in the four districts in 2021, 29.7 million in 2031 and 33.7 million in 2041, against 36.8 million from the simulated-P model of Section 7.2 (Figure 15). The difference comes from the arrivals and emigration, not from movement between the four regions, which is now tiny. The forecast hardly depends on the allocation assumption: 33.65–33.92 million in 2041 for w = 1 to 4. Delhi reaches about 24.5 million.

Fig 15: Forecast 2011-2041, census-data vs simulated model

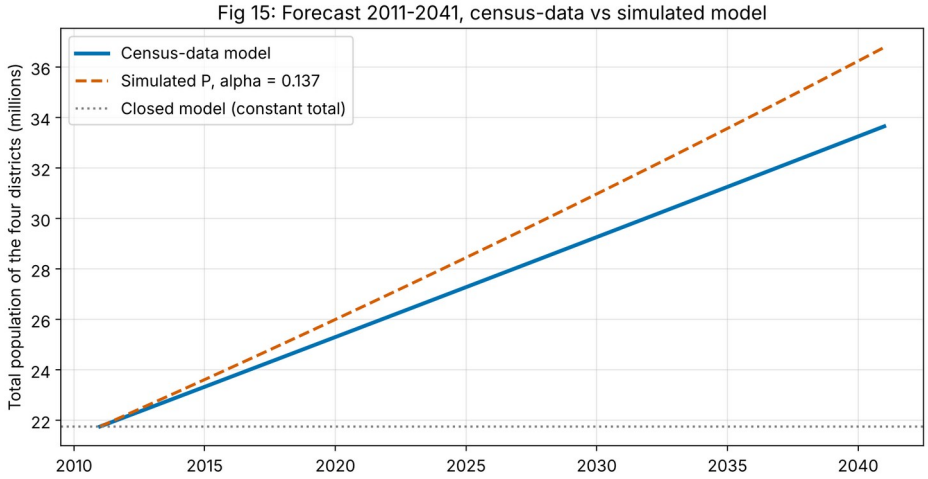


Figure 15: Total population of the four districts, 2011–2041: census-data model, simulated-P model and the closed model.

8 Discussion

8.1 What the numbers mean in plain English

Every year about 5% of Delhi's residents move to the other three regions, but Delhi receives 6–7% of the residents of each of the others. Because Delhi starts with 77% of the people, the movers going into Delhi are few in absolute terms compared with the many movers going out, so Delhi's share falls steadily until outflow equals inflow at 55.5%. The other three regions grow towards 17.5%, 16.7% and 10.3%. After about 32 years the distribution is within 1% (in total distance) of the steady state, so the long-term picture is reached within a generation or two.

We must be careful: in this model the *number* of people living in Delhi falls from 16,787,941 to 12,207,751 after 30 years. Real Delhi has not behaved this way, because births and migration from other states more than compensate. This is an artefact of the closed-system assumption and the simulated probabilities, not a prediction. Section 7 removes the closed-system assumption.

8.2 Limitations

- **Closed system (Sections 4–6):** no births, deaths, or movement into or out of the four regions. Section 7 relaxes this, but its emigration rates are simulated.
- **Constant P:** real movement probabilities change with the economy, policy and infrastructure.
- **Memoryless:** a person's next move is assumed to depend only on where they live now, not on how long they have lived there.
- **Simulated data:** the transition probabilities of Sections 4–7.4 are not measured. Section 7.5 replaces them with Census D-02 flows, but D-02 names only the origin state for some flows, which we allocate by population; and D-02 counts only people who stayed, so it slightly undercounts moves.
- **Constant retention and identical people:** all residents of a region are treated the same.
- **Exact calibration:** in Section 7, four unknown arrival numbers are fitted to four census values, so the fit is perfect by construction and is not an out-of-sample test.
- **Old data:** Census 2011 is the latest census available; the SRS growth rates are state averages applied to districts.

8.3 What if P is not regular?

The steady-state theorem needs regularity. If a region has retention 1 (an absorbing state), people who enter it never leave. In our 3-state example with such a state, starting from state 1 the population after 200 steps is (0.000003, 0.000004, 0.999993): everybody ends in the absorbing state, and the steady state is not driven by inflow–outflow balance. A periodic chain such as P = [[0, 1],[1, 0]] has eigenvalues 1 and −1; starting from (1, 0) the vector flips between (1, 0) and (0, 1) forever and has no limit. For these chains the $|\lambda_2| < 1$ argument fails, so the result of Section 2.3 cannot be used. Our matrix is strictly positive, so it is regular.

8.4 What real data would change

Section 7.5 shows what real data change: Census D-02 flows are about 100 times smaller than our simulated entries, so retention is above 0.998 everywhere and the closed model barely moves on human time scales. What matters in practice are births, deaths and migration across the boundary, which is why the open model is the useful one. District-to-district origin data (not only origin state), yearly updates and the 2027 census would let us estimate P fully and test forecasts on data the model has not seen.

9 Conclusion

We built a regular, column-stochastic transition matrix for four NCR regions, projected the population by matrix multiplication and found the steady state (Delhi 55.48%, Gurugram 17.47%, Noida 16.73%, Faridabad 10.32%) by three independent methods that agree. Diagonalisation shows that the $\lambda = 1$ eigenvector survives while the other components decay at the rates 0.8862^k, 0.8638^k and 0.8600^k, and $|\lambda_2|$ predicts the convergence time well. The scenarios show that small changes in yearly probabilities shift the long-run shares considerably, whereas ±2% random errors change them by at most 1.2 percentage points. The main limitation is that the transition probabilities are simulated; the project demonstrates the method, and real migration data would be needed for real predictions. Extending the model to births, deaths and migration and fitting it to Census 2001 and 2011 showed that the simulated P moves about seven times too many people ($\alpha = 0.137$) and gave a total of about 36.8 million people in the four districts by 2041, growing 0.63% a year in the long run. Building P from the Census 2011 migration table showed that real movement between the regions is about 100 times smaller still, so the closed steady state is a mathematical result rather than a forecast; the census-data open model passes an independent check against the Economic Survey of Delhi and projects about 33.7 million people by 2041.

10 References

1. D. Poole, Linear Algebra: A Modern Introduction (course textbook; chapter on Markov chains).
2. D. G. Zill, Advanced Engineering Mathematics (for matrices and eigenvalue methods).
3. Census of India 2011, Office of the Registrar General & Census Commissioner, India: district totals for NCT of Delhi, Gurgaon, Gautam Buddha Nagar and Faridabad (also the 2001 totals used for calibration in Section 7).
4. Census of India 2011, Table D-02: Migrants classified by place of last residence, sex and duration of residence in the place of enumeration (Haryana, NCT of Delhi, Uttar Pradesh), and Primary Census Abstract 2011.
5. Directorate of Economics & Statistics, Government of NCT of Delhi. Economic Survey of Delhi 2014–15, Chapter 2: Demographic Profile (net migration and vital statistics 2001–2012).
6. Reserve Bank of India, Handbook of Statistics on Indian States, Tables 8–9: state-wise birth and death rates (SRS), 2001–2011.
7. Office of the Registrar General, India. Sample Registration System Bulletin 2020 (Vol. 55, No. 1): crude birth and death rates by state.
8. R. A. Horn and C. R. Johnson, Matrix Analysis, 2nd ed., Cambridge University Press (Perron–Frobenius theorem and Wielandt's bound for primitive matrices).
9. Python 3, NumPy, Matplotlib, pandas and seaborn software documentation (used for all computations and figures).
10. Project code, tests and interactive dashboard: github.com/CRYPT-hack/population-prediction- (tag v1.0-submission holds the original version of this report).

Individual contribution (placeholder – edit before submission)

Member	Roll No.	Contribution (edit)
Mansi Awasthi	250450	e.g. Mathematical background and 2 × 2 example
Prince Kumar Yadav	250500	e.g. Python code, figures and results
Shobhit Rai	250557	e.g. Scenario and sensitivity analysis
Ishank Chauhan	250406	e.g. Report editing, data and coordination

Appendix A: Core Code Listing (markov_model.py)

The full project has these files: inputs.py (all data), markov_model.py (the linear algebra, listed below), project_code.py (closed model, Sections 4–6), open_system.py (Sections 7.1–7.4), census_data.py and census_model.py (Census D-02 model, Sections 7.5–7.7), run_all.py (runs everything) and 38 automated tests. They are in the GitHub repository cited in Section 10; python run_all.py regenerates every number and figure in this report.

```
"""
Reusable linear-algebra core for the NCR population model.

Every function is pure (no printing, no globals) so it can be unit-tested and reused by
`project_code.py` (closed system), `open_system.py` (births, deaths, migration) and the tests.
Convention: column-stochastic matrices, x_{k+1} = P x_k.
"""
from __future__ import annotations

import math
from dataclasses import dataclass

import numpy as np


# ---------------------------------------------------------------------------------------------
# Validation
# ---------------------------------------------------------------------------------------------
def validate_stochastic(P: np.ndarray, tol: float = 1e-9) -> None:
    """Raise ValueError (with a readable reason) unless P is a square column-stochastic matrix."""
    P = np.asarray(P, dtype=float)
    if P.ndim != 2 or P.shape[0] != P.shape[1]:
        raise ValueError(f"P must be square, got shape {P.shape}")
    if not np.isfinite(P).all():
        raise ValueError("P contains NaN or infinite entries")
    if (P < -tol).any():
        i, j = np.argwhere(P < -tol)[0]
        raise ValueError(f"P[{i},{j}] = {P[i, j]} is negative")
    sums = P.sum(axis=0)
    bad = np.where(abs(sums - 1) > tol)[0]
    if bad.size:
        raise ValueError(f"column(s) {bad.tolist()} sum to {sums[bad].round(6).tolist()}, not 1")


def regularity_index(P: np.ndarray) -> int | None:
    """Smallest k with P^k > 0 (entrywise), or None if P is not regular.

    By Wielandt's theorem a primitive n x n matrix satisfies P^k > 0 for some k <= (n-1)^2 + 1,
    so checking up to that bound is a PROOF, not a guess (the original code looped to an arbitrary 50/200).
    Only the zero pattern matters, so we work with booleans (no round-off can create fake zeros).
    """
    n = P.shape[0]
    B = np.asarray(P) > 0
    Bk = B.copy()
    for k in range(1, (n - 1) ** 2 + 2):
        if Bk.all():
            return k
        Bk = (B.astype(int) @ Bk.astype(int)) > 0
    return None


# ---------------------------------------------------------------------------------------------
# Steady state - three independent methods
# ---------------------------------------------------------------------------------------------
def steady_state_eig(P: np.ndarray) -> np.ndarray:
    """Eigenvector for the eigenvalue closest to 1, normalised to sum 1."""
    w, V = np.linalg.eig(P)
    i = int(np.argmin(abs(w - 1)))
    if abs(w[i] - 1) > 1e-8:
        raise ValueError(f"no eigenvalue equal to 1 (closest is {w[i]})")
    v = V[:, i]
```

```
    v = v / v.sum()                      # complex-safe normalisation, removes arbitrary phase
    if np.max(abs(v.imag)) > 1e-10:
        raise ValueError("steady-state eigenvector is not real")
    return v.real


def gauss_solve(A: np.ndarray, b: np.ndarray, log: list | None = None, tol: float = 1e-14) -> np.ndarray:
    """Gaussian elimination with partial pivoting + back substitution (course style).

    BUG FIX: the original divided by the pivot without checking it, so a singular system
    (e.g. a non-regular P with several steady states) produced inf/nan silently.
    If `log` is a list, the row-echelon form is appended to it as text lines.
    """
    A = np.asarray(A, dtype=float).copy()
    b = np.asarray(b, dtype=float).copy()
    m = len(b)
    for c in range(m):
        p = c + int(np.argmax(abs(A[c:, c])))
        if abs(A[p, c]) < tol:
            raise np.linalg.LinAlgError(f"matrix is singular (no pivot in column {c})")
        A[[c, p]] = A[[p, c]]
        b[[c, p]] = b[[p, c]]
        for r in range(c + 1, m):
            f = A[r, c] / A[c, c]
            A[r, c:] -= f * A[c, c:]
            b[r] -= f * b[c]
    if log is not None:
        log.append("Row-echelon form [A | b] after forward elimination:")
        for r in range(m):
            log.append("   " + "  ".join(f"{A[r, j]:9.5f}" for j in range(m)) + "  |  " + f"{b[r]:9.5f}")
    x = np.zeros(m)
    for r in range(m - 1, -1, -1):
        x[r] = (b[r] - A[r, r + 1:] @ x[r + 1:]) / A[r, r]
    return x


def steady_state_system(P: np.ndarray):
    """Solve (P - I) pi = 0 with the last equation replaced by sum(pi) = 1.

    Returns the augmented system (A, b); solve it with gauss_solve. Replacing one equation is valid because the
    rows of (P - I) are linearly dependent (they sum to the zero row)."""
    n = P.shape[0]
    A = P - np.eye(n)
    A[-1, :] = 1.0
    b = np.zeros(n)
    b[-1] = 1.0
    return A, b


def steady_state_power(P: np.ndarray, k: int = 200) -> np.ndarray:
    """Column 0 of P^k. (All columns are equal once P^k has converged.)"""
    return np.linalg.matrix_power(P, k)[:, 0]


# ---------------------------------------------------------------------------------------------
# Spectrum
# ---------------------------------------------------------------------------------------------
@dataclass
class Eigen:
    values: np.ndarray        # sorted by |lambda| descending
    V: np.ndarray             # columns = eigenvectors (v1 sums to 1, others unit length)
    Vinv: np.ndarray


def eigendecomposition(P: np.ndarray) -> Eigen:
    """P = V D V^-1 with eigenvalues sorted by MODULUS.

    BUG FIX: the original sorted by the real value (np.argsort(-w)). For a matrix with a
    negative eigenvalue such as -0.95 this puts it last, so |lambda_2| (convergence speed)
    was read from the wrong eigenvalue. Sorting by modulus is what the theory needs.
    """
```

```
    w, V = np.linalg.eig(P)
    if np.allclose(w.imag, 0) and np.allclose(V.imag, 0):
        w, V = w.real, V.real
    order = np.lexsort((-np.real(w), -abs(w)))   # by |lambda|, ties broken by real part
    w, V = w[order], V[:, order].copy()
    for j in range(V.shape[1]):
        if j == 0 and abs(V[:, 0].sum()) > 1e-12:
            V[:, 0] = V[:, 0] / V[:, 0].sum()
        else:
            V[:, j] = V[:, j] / np.linalg.norm(V[:, j])
            first = int(np.argmax(abs(V[:, j]) > 1e-12))
            if np.real(V[first, j]) < 0:
                V[:, j] *= -1
    if np.linalg.cond(V) > 1e12:
        raise np.linalg.LinAlgError("P is (numerically) not diagonalisable")
    return Eigen(w, V, np.linalg.inv(V))


def second_eigenvalue_modulus(P: np.ndarray) -> float:
    ev = np.linalg.eigvals(P)
    ev = ev[np.argsort(-abs(ev))]
    return float(abs(ev[1]))


def half_life(lam2: float) -> float:
    """Years for the slowest transient mode to halve."""
    return math.log(0.5) / math.log(lam2)


# ---------------------------------------------------------------------------------------------
# Projection & convergence
# ---------------------------------------------------------------------------------------------
def project(P: np.ndarray, x0: np.ndarray, K: int) -> np.ndarray:
    """Rows are x_0 .. x_K."""
    X = np.zeros((K + 1, len(x0)))
    X[0] = x0
    for k in range(K):
        X[k + 1] = P @ X[k]
    return X


def distance_curve(P: np.ndarray, start: np.ndarray, pi: np.ndarray, kmax: int = 120) -> np.ndarray:
    """L1 distance d_k = sum_i |x_k,i - pi_i| between the share vector and the steady state."""
    x = np.asarray(start, dtype=float)
    d = []
    for _ in range(kmax + 1):
        d.append(np.abs(x - pi).sum())
        x = P @ x
    return np.array(d)


def first_below(dist: np.ndarray, thr: float) -> int | None:
    """First k with dist[k] < thr.

    BUG FIX: the original used int(np.argmax(dist < thr)), which returns 0 (='converged
    immediately') when the threshold is NEVER reached. Now we return None in that case."""
    hits = np.flatnonzero(dist < thr)
    return int(hits[0]) if hits.size else None


def convergence_constant_theory(eig: Eigen, start_share: np.ndarray) -> float:
    """C in d_k ~ C |lambda_2|^k, computed from the eigen-decomposition: C = ||c_2 v_2||_1.

    BUG FIX: the original 'theory' fitted C from the simulated curve itself (C = d_20/|l2|^20),
    so comparing 'theory vs actual' was partly circular. This value uses only V^-1 x_0."""
    c = eig.Vinv @ start_share
    return float(np.abs(c[1] * eig.V[:, 1]).sum())


def years_to_threshold(C: float, lam2: float, thr: float) -> int:
    return max(0, math.ceil(math.log(thr / C) / math.log(lam2)))
```

```
# ------------------------------------------------------------------------------------------
# Sensitivity
# ------------------------------------------------------------------------------------------
def perturb(P: np.ndarray, rel: float, rng: np.random.Generator) -> np.ndarray:
    """Multiply each entry by U[1-rel, 1+rel] and renormalise columns."""
    Q = P * rng.uniform(1 - rel, 1 + rel, size=P.shape)
    return Q / Q.sum(axis=0, keepdims=True)


def sensitivity(P: np.ndarray, n_runs: int = 1000, rel: float = 0.02, seed: int = 42):
    """Steady state and |lambda_2| of `n_runs` randomly perturbed copies of P.

    Vectorised: all perturbed matrices are built as one (n_runs, n, n) array and NumPy decomposes the whole
    stack in one call, with ONE eigen-decomposition per matrix (the original did two: eig for the steady
    state and eigvals for lambda_2). Same random numbers, identical results to the loop, about 10x faster."""
    rng = np.random.default_rng(seed)
    n = P.shape[0]
    Q = P[None] * rng.uniform(1 - rel, 1 + rel, size=(n_runs, n, n))
    Q /= Q.sum(axis=1, keepdims=True)
    w, V = np.linalg.eig(Q)                                  # batched: w (runs, n), V (runs, n, n)
    idx = np.argmin(abs(w - 1), axis=1)
    v = np.take_along_axis(V, idx[:, None, None], axis=2)[:, :, 0]
    shares = np.real(v / v.sum(axis=1, keepdims=True))
    lam2 = np.sort(abs(w), axis=1)[:, -2]
    return shares, lam2


def replace_column(P: np.ndarray, col: int, new_col) -> np.ndarray:
    """Return a copy of P with one column replaced, checking it is still a probability vector."""
    M = np.array(P, dtype=float)
    M[:, col] = new_col
    if abs(M[:, col].sum() - 1) > 1e-12 or (M[:, col] < 0).any():
        raise ValueError(f"new column {col} is not a probability vector (sum = {M[:, col].sum()})")
    return M


# ------------------------------------------------------------------------------------------
# Open system:  x_{k+1} = A x_k + m,   A = G M
# ------------------------------------------------------------------------------------------
def open_system_matrix(P: np.ndarray, emigration: np.ndarray, growth: np.ndarray) -> np.ndarray:
    """A = G M where
         M = P diag(1 - e)   (column j: a fraction e_j leaves NCR, the rest moves according to P)
         G = diag(1 + g)     (natural growth: births - deaths, applied after moving).
    A is non-negative but no longer stochastic: column sums of M are 1 - e_j < 1."""
    e = np.asarray(emigration, dtype=float)
    g = np.asarray(growth, dtype=float)
    if ((e < 0) | (e >= 1)).any():
        raise ValueError("emigration rates must lie in [0, 1)")
    if (g <= -1).any():
        raise ValueError("growth rates must exceed -100 %")
    return np.diag(1 + g) @ P @ np.diag(1 - e)


def homogeneous_matrix(A: np.ndarray, m: np.ndarray) -> np.ndarray:
    """Augmented (n+1)x(n+1) matrix B = [[A, m], [0, 1]] so that [x_{k+1}; 1] = B [x_k; 1].
    This turns the AFFINE update x -> A x + m into a purely LINEAR one (same trick as in graphics)."""
    n = A.shape[0]
    B = np.zeros((n + 1, n + 1))
    B[:n, :n] = A
    B[:n, n] = m
    B[n, n] = 1.0
    return B


def project_open(A: np.ndarray, m: np.ndarray, x0: np.ndarray, K: int) -> np.ndarray:
    X = np.zeros((K + 1, len(x0)))
    X[0] = x0
    for k in range(K):
        X[k + 1] = A @ X[k] + m
```

```
    return X


def geometric_sum(A: np.ndarray, K: int) -> np.ndarray:
    """S_K = I + A + ... + A^(K-1)."""
    n = A.shape[0]
    S, Ak = np.zeros((n, n)), np.eye(n)
    for _ in range(K):
        S += Ak
        Ak = Ak @ A
    return S


def calibrate_immigration(A: np.ndarray, x_start: np.ndarray, x_end: np.ndarray, years: int) -> np.ndarray:
    """Solve for the constant yearly immigration m that carries x_start to x_end in `years` steps:

        x_end = A^K x_start + S_K m     =>    S_K m = x_end - A^K x_start

    A 4x4 linear system, solved with our own Gaussian elimination."""
    rhs = x_end - np.linalg.matrix_power(A, years) @ x_start
    return gauss_solve(geometric_sum(A, years), rhs)


def perron(A: np.ndarray):
    """Dominant eigenvalue rho and its eigenvector (normalised to sum 1) of a non-negative matrix.
    Perron-Frobenius: for a positive matrix rho is real, simple, and its eigenvector is positive."""
    w, V = np.linalg.eig(A)
    i = int(np.argmax(abs(w)))
    v = V[:, i] / V[:, i].sum()
    return float(w[i].real), v.real


def affine_fixed_point(A: np.ndarray, m: np.ndarray) -> np.ndarray | None:
    """x* = (I - A)^-1 m if rho(A) < 1 (population settles); None if rho(A) >= 1 (it keeps growing)."""
    rho, _ = perron(A)
    if rho >= 1:
        return None
    return np.linalg.solve(np.eye(A.shape[0]) - A, m)


# ---------------------------------------------------------------------------------------------
# Calibration of mobility against the census
# ---------------------------------------------------------------------------------------------
def scale_mobility(P: np.ndarray, alpha: float) -> np.ndarray:
    """P_alpha = I + alpha (P - I).  alpha = 1 gives P, alpha = 0 means nobody moves.

    Columns still sum to 1 and P_alpha pi = pi for the SAME steady state pi, while every
    eigenvalue maps to 1 + alpha (lambda - 1): less mobility -> same destination, slower journey."""
    n = P.shape[0]
    return np.eye(n) + alpha * (np.asarray(P, dtype=float) - np.eye(n))


def calibrate_mobility(P, emigration, growth, x_start, x_end, years, tol=1e-6):
    """Largest alpha in [0, 1] for which the census change x_start -> x_end can be explained
    with NON-NEGATIVE immigration into every region (bisection; min(m) decreases with alpha).

    Returns (alpha, m, A). Raises ValueError if even alpha = 0 needs negative immigration."""
    def m_of(a):
        A = open_system_matrix(scale_mobility(P, a), emigration, growth)
        return calibrate_immigration(A, x_start, x_end, years), A

    m1, A1 = m_of(1.0)
    if m1.min() >= 0:
        return 1.0, m1, A1
    m0, _ = m_of(0.0)
    if m0.min() < 0:
        raise ValueError("census change needs negative immigration even with zero internal mobility")
    lo, hi = 0.0, 1.0
    while hi - lo > tol:
        mid = (lo + hi) / 2
        if m_of(mid)[0].min() >= 0:
```

```
            lo = mid
        else:
            hi = mid
    m, A = m_of(lo)
    return lo, m, A


# ---------------------------------------------------------------------------------------------
# Calibrating emigration with Newton's method
# ---------------------------------------------------------------------------------------------
def calibrate_emigration(P, growth, m, x_start, x_end, years, e0=None, tol=1e-12, max_iter=50):
    """Find emigration rates e (one per region) such that x_start grows into x_end in `years` steps of
    x -> A(e) x + m,  A(e) = diag(1+g) P diag(1-e).

    x_end depends non-linearly on e, so we use Newton's method: at each step build the Jacobian
    J[i, k] = d x_end[i] / d e_k by finite differences and solve the 4x4 linear system J de = -f(e)
    with our own Gaussian elimination. Returns (e, iterations)."""
    def f(e):
        return project_open(open_system_matrix(P, e, growth), m, x_start, years)[-1] - x_end
    n = len(x_start)
    e = np.full(n, 0.005) if e0 is None else np.array(e0, float)
    for it in range(1, max_iter + 1):
        f0 = f(e)
        J = np.zeros((n, n))
        for k in range(n):
            h = 1e-7
            ek = e.copy(); ek[k] += h
            J[:, k] = (f(ek) - f0) / h
        de = gauss_solve(J, -f0)
        e = e + de
        if np.max(abs(de)) < tol:
            return e, it
    raise RuntimeError("Newton iteration for emigration did not converge")
```

Appendix B: Hand-Calculation Sheet

B.1 Population of Gurugram after one year (use a calculator)

x_1[Gurugram] = Σ_j P[Gurugram][j] · x_0[j] (row 2 of P times the initial vector).

From region j	P[Gurugram][j]	x_0[j]	Product
Delhi	0.020	16,787,941	335,758.8
Gurugram	0.890	1,514,432	1,347,844.5
Noida	0.030	1,648,115	49,443.4
Faridabad	0.030	1,809,733	54,292.0
Total			**1,787,338.7**

Check: the same row sum is the second entry of P x_0 in Python.

B.2 Eigenvalues of P using trace and structure

The trace of P is 3.610, and the eigenvalues add up to the trace: 1.0000 + 0.8862 + 0.8638 + 0.8600 = 3.610. This is a quick check that the printed eigenvalues are right.

B.3 The 2 × 2 example in full

1. T = [[0.9, 0.2],[0.1, 0.8]]. Trace = 1.7 and determinant = 0.9×0.8 − 0.2×0.1 = 0.70.
2. Characteristic equation: $\lambda^2 - 1.7\lambda + 0.70 = 0$, i.e. $(\lambda - 1)(\lambda - 0.7) = 0$.
3. $\lambda = 1$: solve $(T - I)v = 0$. Row 1: $-0.1a + 0.2b = 0 \Rightarrow a = 2b$. Take $v_1 = (2, 1)$; divide by the sum 3 to get $\pi = (2/3, 1/3)$.
4. $\lambda = 0.7$: $(T - 0.7I)v = 0$. Row 1: $0.2a + 0.2b = 0 \Rightarrow b = -a$. Take $v_2 = (1, -1)$.
5. V = [[2, 1],[1, −1]], det V = −3, V^{-1} = [[1/3, 1/3],[1/3, −2/3]], D = diag(1, 0.7).
6. $T^n = V D^n V^{-1}$. As $n \to \infty$, $D^n \to$ diag(1, 0), so $T^n \to$ [[2/3, 2/3],[1/3, 1/3]]: every column is π.
7. Start $x_0 = (0.5, 0.5)$: $x_1 = (0.9\times0.5 + 0.2\times0.5, 0.1\times0.5 + 0.8\times0.5) = (0.55, 0.45)$. Then x_2 = (0.585, 0.415) and x_3 = (0.6095, 0.3905), approaching (0.6667, 0.3333).

B.4 Steady state of P by hand (outline)

Solve $(P - I)\pi = 0$ with $\pi_1 + \pi_2 + \pi_3 + \pi_4 = 1$: write the augmented matrix, replace the last equation by the row of ones, and apply row operations (forward elimination, then back-substitution). The program prints the row-echelon form it obtains in results.txt (Section 3), so each row operation can be checked against it.